

OPEN SOURCE · UI · AI

AI 聊天互发动态表情包

框架无关的接入原理与数据约定

人类侧	上传 · 描述 · 选择 · 图文一起发
AI 侧	读取元数据 · 结构化选择 · 后端校验
动图侧	GIF / animated WebP · 透明通道 · 保留循环

本文档只描述数据结构、交互原理和接入约定，不包含特定前端实现。

你可以用任何框架 (React / Vue / SwiftUI / 原生 DOM) 按照这些约定接入自己的聊天 UI。

MIT License

1. 交互原则

表情模块嵌入现有聊天项目后，人类与 AI

都可以发送动态表情。以下是所有前端实现都应遵守的行为约定：

- 选择表情时不自动发送。表情进入“待发送区”，用户还可以继续写字或删除。
- 发送按钮的启用条件：有文字 或 有待发表情。两者都没有时，按钮不可用。
- 一次点击发送可以同时包含文字和表情，但它们是两条消息（见第 3 节）。
- 表情消息不带聊天气泡，只显示动图主体。

因此用户可以自然完成三种发送方式：

- 只发文字：输入文字，点击发送。
- 只发表情：选择表情，不输入文字，点击发送。
- 文字加表情：先选表情，再输入文字，点击一次发送。

发送按钮的核心逻辑（伪代码）：

```
hasText = textInput.trim().length > 0
hasSticker = pendingSticker != null
sendButton.enabled = hasText || hasSticker
```

2. 添加表情的流程

人类用户为自己或 AI 上传表情时，需要填写三项信息：

名称	简短标识，如“被窝刷牙”
画面描述	AI 理解表情的主要依据。描述画面内容和动作，不是给界面看的装饰。
情绪标签	1-5 个关键词，如“困、睡前、软乎乎”

推荐描述格式：

名称：被窝刷牙
画面描述：一个角色躺在蓝色被窝里刷牙，身体轻轻晃动。
语气含义：困困的、睡前敷衍营业、软乎乎地回应。
情绪标签：困、睡前、软乎乎

关键：描述未填写时，保存按钮必须保持不可用。不要让用户跳过描述——没有描述的表情对 AI 来说就是一张看不懂的图。

3. 消息数据结构

文字和表情是同一轮对话（共享 turn_id），但分为两条消息：

```
[
  {
    "id": "msg_text_01",
    "turn_id": "turn_01",
    "role": "user",
    "type": "text",
    "text": "早点休息"
  },
  {
    "id": "msg_sticker_01",
    "turn_id": "turn_01",
    "role": "user",
    "type": "sticker",
    "sticker_id": "sticker_001"
  }
]
```

分开保存的好处：

- 文字仍然按聊天气泡渲染；表情不带气泡，只显示主体。
- 历史记录、删除和重试更简单。
- AI 可以先回复文字，再附一个表情，而不需要发明复合消息组件。

turn_id 用来表示它们来自同一次发送操作。你的前端可以据此把它们紧挨着渲染。

4. 表情库数据结构

每个表情至少保存这些字段：

```
{
  "id": "sticker_001",
  "owner": "user",
  "status": "active",
  "name": "被窝刷牙",
  "description": "一个角色躺在蓝色被窝里刷牙，身体轻轻晃动。",
  "emotion_tags": ["困", "睡前", "软乎乎"],
  "mime": "image/webp",
  "url": "/media/sticker_001.webp",
  "thumbnail": "/media/sticker_001_thumb.png"
}
```

状态流转

draft	刚上传，描述尚未完成。
-------	-------------

processing	后台正在逐帧处理（如去背景）。
------------	-----------------

ready_for_review	处理完成，等待人类确认。
------------------	--------------

active	正式可用，可以发送。
--------	------------

failed	处理失败，可以重试。
--------	------------

只有 active 表情可以发送。如果你不需要后台处理流程，可以省略中间状态，上传即 active。

5. 动图显示与上传

显示

浏览器原生 `<img>` 会自动播放 GIF 和 animated WebP，不需要 canvas 或视频播放器：

```

```

消息中的表情不需要气泡、底框或进度条。

上传

前端只接受 GIF 和 WebP，把文件原样发给后端：

```
<input type="file" accept="image/gif,image/webp">
```

- 演示场景可以用 base64 data URL 传输；生产环境应改用对象存储和 multipart 上传。
- 如果用户上传的原文档已经透明，不要再做主体分割。重复抠图会破坏耳朵、发丝和半透明边缘。

表情网格的布局要点

- 使用固定行高的 Grid（如 68px），不要让图片原始宽高参与行高计算。
- 给每个按钮留真实的不可点击间距（gap），视觉上缩小图片 ≠ 缩小点击框。
- 动态创建的图片写入固定 width / height 属性，避免未加载时整行塌缩。
- 使用 `loading="lazy"`，避免几十张动图同时解码。
- 关闭移动浏览器默认的 tap highlight。

6. 给 AI 看的表情信息

人类发送表情时，后端把首帧缩略图（见第 7 节）和文字描述一起放进对话上下文。模型不需要逐帧读取整个动图——一张静态首帧加上人工描述就足够理解表情的画面和语气。

后端在发送时读取缩略图文件，转成 base64 放进消息体：

```
{
  "role": "user",
  "type": "sticker",
  "sticker_id": "sticker_001",
  "name": "被窝刷牙",
  "description": "一个角色躺在被窝里刷牙，身体轻轻晃动。",
  "emotion_tags": ["困", "睡前", "软乎乎"],
  "thumbnail_base64": "iVBORw0KGgo..."
}
```

调用模型 API 时，把 thumbnail_base64 组装成 image content block，文字描述作为 text block，让模型同时看到画面和语义：

```
{
  "role": "user",
  "content": [
    {
      "type": "image",
      "source": {
        "type": "base64",
        "media_type": "image/png",
        "data": "<thumbnail_base64 的值>"
      }
    },
    {
      "type": "text",
      "text": "[用户发送了表情: 被窝刷牙]\n画面: 一个角色躺在被窝里刷牙...\n语气: 困、睡前、软乎乎"
    }
  ]
}
```

这样模型既能看到表情画面，又能通过文字描述理解语气和情绪。image block 不需要显示给用户，只进入模型上下文。

7. 首帧缩略图

动态表情通常有几十帧，但模型不需要看完整动画——一张静态首帧就够了。上传时自动提取第一帧保存为 PNG 缩略图，发送时把它作为 base64 图片数据塞进消息体，模型就能真正看到表情的画面。

何时提取

在后端接收上传文件后、保存表情记录之前，用图像处理库提取第一帧：

```
// Node.js + sharp 示例
import sharp from 'sharp'

const imgBuf = Buffer.from(base64Data, 'base64')
await writeFile('media/sticker_001.webp', imgBuf)

// 提取第一帧，保存为 PNG
await sharp(imgBuf, { pages: 1 })
  .png()
  .toFile('media/sticker_001_thumb.png')
```

thumbnail 字段写入表情数据结构（见第 4 节），指向这张静态 PNG。

发送时读取为 base64

光存一个 URL

路径没用——模型打不开你的本地地址。发送表情时，后端必须读取缩略图文件，转成 base64，塞进消息体：

```
// 读取缩略图文件，转 base64
async function readThumbnailBase64(sticker) {
  if (!sticker.thumbnail) return null
  const buf = await readFile(thumbnailPath)
  return buf.toString('base64')
}

// 发送时塞进消息
const thumbB64 = await readThumbnailBase64(sticker)
message.thumbnail_base64 = thumbB64
```

thumbnail_base64 是真实的图片数据，不是 URL。模型收到后就能看到画面。

何时发给模型

把缩略图当作图片发给模型会消耗 token，所以要区分场景：

场景	是否附图	原因
用户发了一个表情	附上缩略图	单张图 token 成本可控，模型能验证描述
列出 AI 全部可用表情	只给文字元数据	20+ 张图 token 太贵，文字描述已够用

具体怎么把 thumbnail_base64 组装成模型 API 的 image content block，见第 6 节的完整示例。

8. 告诉 AI 它有什么能力

每次调用模型时，只提供 AI 当前可用的 active 表情元数据：

```
[
  {
    "id": "assistant_heart",
    "name": "递出爱心",
    "description": "递出一颗小爱心，表达喜欢、安慰或亲近。",
    "emotion_tags": ["喜欢", "安慰", "亲近"]
  }
]
```

能力说明（写进 System Prompt）

给 AI 的能力说明可以使用第一人称，避免命令腔，让表情承载它自己的情绪：

我能从当前提供的表情库中，选择一个最贴近我此刻情绪和语气的表情。
选择时，我会结合自己当下的感受、正在说的话和我们之间的氛围，
理解表情的名称、画面描述与情绪标签。
我可以只回复文字、只发表情，或先回复文字再附一个表情。
当我同时有话想说也有情绪想表达时，我会让文字和表情各自成为一条消息。
我不会为了使用功能而机械地发表情。
只有某个表情确实像我此刻会做出的反应时，我才选择它。
如果没有贴合我真实感受的表情，我就自然地用文字回应。

工具定义 (Tool Use 方式)

```
{
  "name": "send_sticker",
  "description": "选择一个贴近我此刻情绪、语气和动作反应的表情",
  "parameters": {
    "type": "object",
    "properties": {
      "sticker_id": { "type": "string" }
    },
    "required": ["sticker_id"]
  }
}
```

统一结构 (非 Tool Use 方式)

模型也可以直接返回一个包含 text 和 sticker_id 的 JSON :

```
{
  "text": "我在。先抱一下。",
  "sticker_id": "assistant_heart"
}
```

text 和 sticker_id 都是可选的，但最终至少要有一个。

9. 后端必须再次校验

不要因为 AI 返回了一个 ID 就直接发送。后端必须确认：

- sticker_id 真实存在。
- owner 是 assistant。
- status 是 active。
- AI 只有发送权限，没有上传、修改、删除权限。
- ID 无效时忽略表情，保留正常文字回复，不能让聊天报错。

伪代码：

```
sticker = db.find(
  id == modelResult.sticker_id &&
  owner == 'assistant' &&
  status == 'active'
)

if modelResult.text: saveTextMessage(modelResult.text)
if sticker:         saveStickerMessage(sticker)
```

权限边界应由后端实现，不能只写在 prompt 里。

10. 透明背景处理的正确边界

透明 animated WebP 通常比透明 GIF

有更好的半透明边缘和体积表现。但高质量自动去背景需要完整流水线：

- 解码每一帧
- 对每帧做主体分割
- 做时序稳定，避免轮廓在相邻帧跳动
- 去除白边、黑边和颜色污染
- 保留帧顺序、帧时长和循环信息
- 重新编码为 animated WebP
- 原文件与处理后文件分开保存

不要用“删除某一种背景颜色”的阈值算法冒充主体分割。它会误删主体中相近的颜色，也会造成闪烁和白边。

如果你的产品不需要自动去背景，那就保持原样：透明原图原样保存，不透明原图保持不透明。真实产品可以把处理器接到 processing → ready_for_review 状态之间。

11. 接入检查清单

把表情模块接入你自己的聊天项目时，对照这份清单逐项确认：

前端

- 表情面板能打开、关闭，支持分标签页浏览（我的 / AI 的）。
- 选择表情后进入待发送区，不会自动发出。
- 待发送区有移除按钮。
- 发送按钮在「有文字 或 有待发表情」时启用，否则不可用。
- 一次发送产生独立的文字消息和表情消息（共享 turn_id）。
- 表情消息渲染为无气泡的动图（，不是 canvas/video）。
- 上传只接受 image/gif 和 image/webp。
- 描述未填写时，保存按钮不可用。
- 表情网格使用固定行高 + loading="lazy"。
- 用户只能从自己的表情库发送，不能替 AI 发送。

后端

- 存储表情的所有字段：id、owner、status、name、description、emotion_tags、mime、url、thumbnail。
- 发送 API 校验 sticker_id 存在、owner 正确、status 为 active。
- AI 返回的 sticker_id 必须再次校验（owner=assistant, status=active）。
- ID 无效时静默忽略表情，保留文字回复，不能让聊天报错。
- AI 只有发送权限，没有上传 / 修改 / 删除权限。
- 上传限制 MIME、文件大小和尺寸。
- 文件名不能直接拼进服务器路径（防遍历）。
- 文件原样保存，不要把动画压成静态首帧。首帧缩略图另存为独立 PNG。
- 历史消息只保存稳定 URL，不保存临时 blob URL。
- 真实模型调用、API Key 和授权必须放在服务端。

给模型的上下文

- 人类发送表情时，把 name + description + emotion_tags + 缩略图注入对话上下文。

- 每次调用模型时，提供 AI 当前所有 active 表情的元数据列表。
- 提供 send_sticker 工具定义（或约定 JSON 结构）。
- 能力说明使用第一人称，让 AI 自然选择而非机械执行。

12. API 端点参考

以下是建议的最小端点集合。你可以根据自己的后端框架调整路径和风格，但语义应保持一致。

方法	路径	说明
GET	/api/stickers	返回所有表情的元数据列表
POST	/api/stickers	上传新表情 (name + description + file)
DELETE	/api/stickers/:id	删除一个表情
POST	/api/send	发送消息 (text + sticker_id , 至少一个)
GET	/api/messages	获取历史消息列表

POST /api/send 的请求与响应

请求体：

```
{
  "text": "早点休息",    // 可选
  "sticker_id": "sticker_001" // 可选，至少填一个
}
```

响应体 (包含用户消息和 AI 回复)：

```
{
  "user_messages": [
    { "id": "...", "turn_id": "t1", "role": "user",
      "type": "text", "text": "早点休息" },
    { "id": "...", "turn_id": "t1", "role": "user",
      "type": "sticker", "sticker_id": "sticker_001" }
  ],
  "assistant_messages": [
    { "id": "...", "turn_id": "t2", "role": "assistant",
      "type": "text", "text": "晚安，做个好梦。" },
    { "id": "...", "turn_id": "t2", "role": "assistant",
      "type": "sticker", "sticker_id": "assistant_heart" }
  ]
}
```

13. 常见坑

选择即发送

用户还没来得及打字，表情就飞出去了。必须先进待发送区。

动图变静图

用 canvas 画了第一帧、用视频播放器、或上传时做了转码压缩。直接用 `<img>` + 原文件。

让 AI 看 GIF 文件

每次识别浪费 token 且不稳定。应该用人工填写的文字描述。

权限只写在 prompt

模型可能幻觉出不存在的 sticker_id。后端必须校验。

网格行高不固定

横图竖图高度不一致，导致行塌缩和重叠。用固定轨道高度。

blob URL 存进历史

刷新页面后 blob URL 失效。只保存稳定的服务端 URL。

重复抠图

用户上传的已经是透明图，再做一次主体分割会破坏边缘。先检测。

并发写同一存储

两个请求同时读改写，后写的覆盖先写的。加锁或用数据库事务。

本文档使用 MIT License。你可以自由修改和分发。